

HACKATHON FINAL SUBMISSION

Skill Swap Platform

Technical Architecture & System Documentation

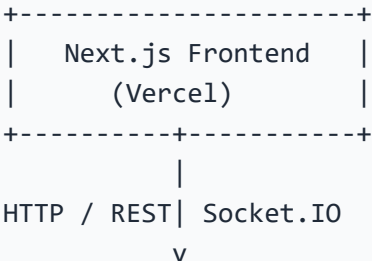
Technical Documentation - Skill Swap Platform

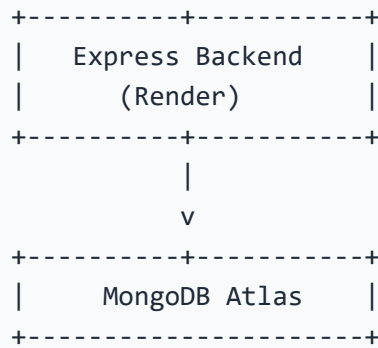
Welcome to the technical documentation for the **Skill Swap Platform**. This document provides an in-depth overview of the application's architecture, database design, algorithmic engines, and API endpoints.

-

System Architecture

The platform is designed as a decoupled monorepo structured into two primary components: 1. **Frontend** (``client/``): A modern **Next.js** application using Tailwind CSS for UI and Zustand for lightweight global state management. Deployed on **Vercel**. 2. **Backend** (``server/``): A RESTful **Node.js/Express** server utilizing Socket.IO for real-time networking and Mongoose for MongoDB data modeling. Deployed on **Render**.





- --

Database Schema & Models

We define four primary Mongoose models to manage the platform's state:

1. User Model (`server/models/userModel.js`)

Stores user profiles, credentials, skills, availability, reviews, and gamification metrics.

- `name` (String, required)
- `email` (String, required, unique)
- `password` (String, required, hashed via bcryptjs)
- `role` (String: `USER`, `ADMIN`)
- `profilePhoto` (String)
- `location` (String)
- `skillsOffered` (Array of Strings) - Skills the user can teach.
- `skillsWanted` (Array of Strings) - Skills the user wants to learn.
- `availability` (String) - Plain text description or JSON string of slot configurations.
- `xp` (Number, default: 0) - Experience points.
- `level` (Number, default: 1) - User's level.
- `badges` (Array of Strings) - Unlocked awards.
- `isPublic` (Boolean, default: true) - Discoverable in explore/matches.
- `reviews` (Array) - Nested review sub-documents containing `reviewerName`, `rating`, `comment`, and `createdAt`.

2. Swap Request Model

(`server/models/swapRequestModel.js`)

Manages the lifecycle of a knowledge exchange between two users.

- `senderId` (ObjectId, ref: 'User') - The user proposing the swap.
- `receiverId` (ObjectId, ref: 'User') - The target user.
- `offeredSkills` (Array of Strings) - What the sender offers to teach.
- `requestedSkills` (Array of Strings) - What the sender wants to learn.
- `message` (String) - Introduction note.
- `status` (String: `PENDING`, `ACCEPTED`, `REJECTED`, `COMPLETED`)
- `proposedTime` (Date) - Session date agreed upon via smart scheduling.

3. Feedback Model (`server/models/feedbackModel.js`)

Stores completed transaction reviews to calculate reputation scores.

- `swapRequestId` (ObjectId, ref: 'SwapRequest')
- `reviewerId` (ObjectId, ref: 'User')
- `recipientId` (ObjectId, ref: 'User')
- `rating` (Number, 1-5)
- `comment` (String, required)

4. Message Model (`server/models/messageModel.js`)

Manages real-time chat messages between swap partners.

- `senderId` (ObjectId, ref: 'User')
- `receiverId` (ObjectId, ref: 'User')
- `content` (String, required)
- `createdAt` (Date)
- --

Algorithmic Engines

1. AI Match Engine (`server/services/matchService.js`)

The matching engine evaluates compatibility dynamically through a multi-dimensional scoring algorithm:

- **Skill Intersection Score (70%):**

- Checks if a reciprocal swap is possible: User A's `skillsOffered` matches User B's `skillsWanted` AND User B's `skillsOffered` matches User A's `skillsWanted` (+70 points).
- One-way match (+40 points).
- **Context Compatibility Score (20%):**
- Compares locations: matches in the same city receive a bonus.
- Analyzes availability string matching (e.g. both free on "Weekends" or "Evenings").
- **Trust & Reputation (10%):**
- Factored by reviews, rating averages, and user levels.
- **Natural Language Explanations:**
- Synthesizes matches into readable sentences: `*"Perfect reciprocal match! You can teach them React while they teach you UI Design."*`

2. Gamification & XP System (`server/services/xpService.js`)

Designed to incentivize continuous teaching and feedback:

- **XP Triggers:**
- Completing a Swap: **+50 XP** to both partners.
- Leaving Feedback: **+10 XP**.
- **Level Progression:**
- Level is calculated dynamically: ``level = Math.floor(Math.sqrt(xp / 100)) + 1``.
- **Badges Unlocked Triggers:**
- `*First Swap*`: Unlocked upon completing 1 swap.
- `*Top Rated*`: Unlocked when average rating is ≥ 4.8 with at least 3 reviews.
- `*Expert Teacher*`: Unlocked when completed swaps count ≥ 5 .
- --

Smart Scheduling System

Instead of coordinating schedules via email: 1. Under an active swap, a user proposes a session time. 2. The system saves the `proposedTime` in the `SwapRequest` document. 3. Next.js fetches this state and displays the

sessions inside the unified **Calendar View** (`client/src/app/calendar/page.js``), helping users avoid double-booking slots.

- --

API Routes Reference

Authentication

- `POST /api/auth/register`` - Create user profile.
- `POST /api/auth/login`` - Obtain JWT Token.
- `POST /api/auth/logout`` - Clear session.
- `GET /api/auth/me`` - Fetch authenticated user details.

Users & Directory

- `GET /api/users/search`` - Query mentors by skill, location, or availability.
- `GET /api/users/matches`` - Generate AI compatible matches list.
- `GET /api/users/leaderboard`` - Fetch sorted global user ranks.
- `GET /api/users/:id`` - View public profile, reviews, and badges.

Swap Management

- `POST /api/swaps/request`` - Create a swap request.
- `GET /api/swaps/my-requests`` - Fetch all incoming & outgoing requests for the logged-in user.
- `PUT /api/swaps/:id/accept`` - Accept an incoming request.
- `PUT /api/swaps/:id/reject`` - Reject an incoming request.
- `DELETE /api/swaps/:id/cancel`` - Cancel a pending outgoing request.
- `PUT /api/swaps/:id/schedule`` - Propose a scheduling session.
- `POST /api/swaps/:id/complete`` - Complete a swap, save rating, and trigger XP.